

Dynamic parallelism and asynchronous operation
Advanced computer architecture

(B-KUL-JLI1QS)

March 28, 2024



Session Date:	March 22, 2024
Partners:	Emma Dejagere
	Brecht Van de Sijpe
Class:	MElsw
Instructor:	Alex Marinsek

1 Introduction

In this labsession we revisited the reduction we discussed in lab two. We expanded this by performing a summation and a multiplication and by calculating the minimum and the maximum element of an array.

2 Method

The code implements a synchronous and asynchronous approach to call four kernels in which the minimum, maximum, sum and product of elements in an array is calculated. The implementation of these four kernels is identical to the reduction implementation of lab two.

In the synchronous version each kernel is called after the other and after each kernel call *cudaDeviceSynchronize* is called for synchronisation. This method leads to the situation where the preparation of the array is done in synchronous with the kernel-calculations. In the asynchronous version for each kernel a cudaStream is allocated and the memory is copied asynchronously. This leads to a program where the array preparation is done asynchronously from the kernel execution. At the end of the 4 kernel calls each stream is synchronized with *cudaStreamSynchronize*.

3 Results

The results show that the asynchronous version is faster than the synchronous version. This effect is better seen in figure 2 on the RTX 3060.

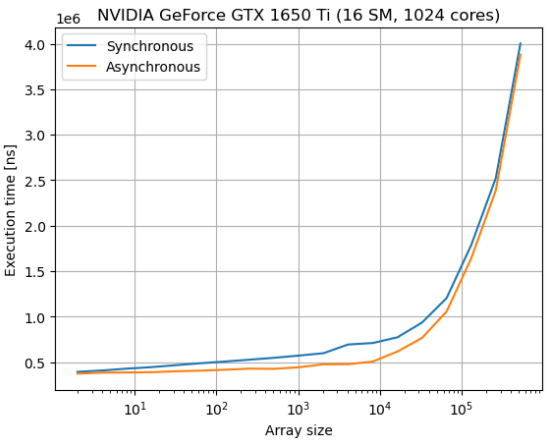


Figure 1: Synchronous vs asynchronous on NVIDIA GeForce GTX 1650 Ti

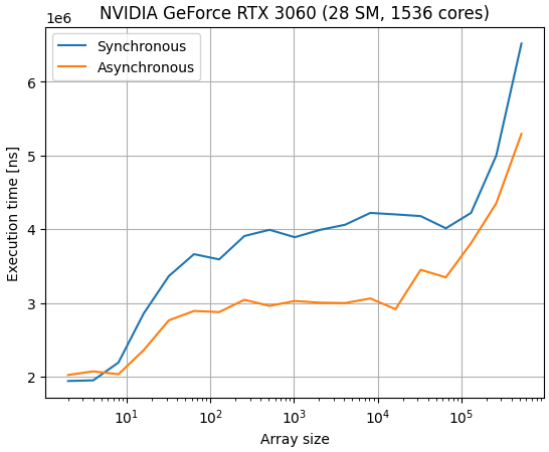


Figure 2: Synchronous vs asynchronous on NVIDIA GeForce RTX 3060

4 Conclusion

We can conclude that synchronous preparation of the array is slower than asynchronous preparation. This is due to the fact that while the preparations for the next kernel are done, the other kernel can already execute.